Flask Task API Guided Python Lab

IN432 Advanced Web Technologies Session 2

Lebanese University Master M1 Dr Zaher Yamak

This tutorial introduces the Python needed to extend a Flask API. Read each short explanation, run its example, then complete a small part of the application.

The task you will follow

In our teaching scenario, Lina is preparing a project presentation. Her API can already read tasks. You will let her create Prepare project proposal, mark it complete, reject an invalid update without changing the task, and delete it.

Your route through the core lab

The Blueprint comparison, PUT and PATCH comparison, request boundary checks and restart experiment follow the core pages and can be completed after class.

Use the student starter for this edition. The earlier starter has larger TODO blocks. Work on your own laptop; a neighbour may help you read an error, while you keep your own code and explanations.

Step 1 Prepare the Python environment

Extract IN432_Session2_Student_Starter.zip. Open the extracted IN432_Session2_Starter folder in your editor.

Check your working folder

The terminal must be inside the folder containing requirements.txt, api_client.py and the task_api directory. Use your editor command Open in Integrated Terminal, or open a terminal and use cd to enter that folder.

```
python --version
python -m venv .venv
```

Use Python 3.10 or newer. If your installed launcher is python3 or py, use that name instead of python when creating the environment. After activation, python should refer to the environment interpreter.

A virtual environment is a project folder containing its Python interpreter and installed packages. It keeps this lab installation separate from other projects. The -m option asks Python to run a module such as venv or pip.

Activate the environment in each terminal

Your terminal	Command
Windows Command Prompt	.venv\Scripts\activate.bat
Windows PowerShell	.venv\Scripts\Activate.ps1
macOS or Linux	source .venv/bin/activate

```
python -m pip install -r requirements.txt
```

requirements.txt lists the Flask package version used by this lab. pip installs it in the active environment. Install once; activate the environment again whenever you open another terminal.

Step 1 Run the API and its client

Terminal A runs Flask

```
python -m flask --app task_api run --debug
```

Keep this terminal open. --app task_api selects the supplied Python package; Flask finds its create_app function. run starts the local development server. --debug reloads it after code edits and is for local development.

Wait for the server address, normally http://127.0.0.1:5000. 127.0.0.1 means your own laptop. If the port is occupied, the error reference page gives matching server and client commands for port 5001.

Terminal B sends requests

Open a second terminal in the same starter folder and activate the same environment. Enter each command as one terminal line.

```
python api_client.py GET /api/health
python api_client.py GET /api/tasks
python api_client.py GET /api/tasks/1
python verify_api.py --baseline
```

api_client.py is a supplied program: GET is its method argument and /api/tasks/1 is its path argument. It sends a real HTTP request to Terminal A. You do not write this client today.

Program	Server needed	What to look for
api_client.py	Yes	HTTP status, headers and JSON body
verify_api.py --baseline	No	4 tests and OK

The baseline checker creates its own applications inside the Python process. Its GET checks work before you finish any TODO. A fresh teaching store contains task IDs 1, 2 and 3.

Step 2 Python basics

Read Python values and dictionaries

Before editing Flask, run the supplied Python example in Terminal B. It needs no server. You can keep Terminal A running.

```
python python_warmup.py
```

Open python_warmup.py in your editor. Start with sections A and B; later pages explain its other sections. A .py file contains Python statements. You run it directly with python filename.py; there is no javac step in this lab.

```
title = "  Prepare project proposal  "
done = False
payload = {"title": title, "done": done}
print(payload["title"])
print(payload["done"])
```

The assignment title = ... gives a name to a string value. No String or char declaration is written. Values still have types: str is text, bool is a boolean and int is an integer.

payload is a dictionary, called dict. A dictionary stores key and value pairs, like a Java Map. payload["title"] retrieves the value under the title key. These braces contain data; Python code blocks use indentation.

Meaning	JSON payload file	Python code
True value	true	True
False value	false	False
No value	null	None
A string containing a word	"false"	"false"

In this API, Flask converts JSON objects into Python dictionaries. The string "false" is text; it is different from a boolean False. Printed Python dictionaries can use single quotes and capitalized booleans; HTTP JSON uses JSON syntax.

Try Change done to True in the warm up, run it again, then restore False. Explain which value changed and which dictionary key stayed the same.

Use indentation and test key presence

In Java and C, braces delimit many blocks. In Python, a colon starts an if or function block and indentation marks its statements. Use four spaces per indentation level and keep the editor indentation consistent.

```
payload = {"done": False}

if "done" in payload:
    print("The client supplied done")
    print(payload["done"])

print("After the if")
```

Read the if condition as: does the dictionary contain a key named done? The two indented print statements belong to the if. The last print has returned to the left edge, so it is outside the block.

Expected output

```
The client supplied done
False
After the if
```

Presence and value answer different questions

Expression	Question	Result here
"done" in payload	Was the key supplied	True
payload["done"]	What value was supplied	False

if payload["done"] would enter its block only for a true value. It would skip a supplied False. Our validator must test presence first so that False remains an accepted update.

Try In section C of python_warmup.py, identify which print depends on the if and which runs afterward. Predict before running. Do not type >>> or shell prompt characters into a .py file.

Define and call small functions

A Python function is similar to a C function or a Java method for this lab: it receives arguments, executes statements and can return a result. def introduces its definition.

```
def clean_title(value):
    return value.strip()

result = clean_title("  Prepare slides  ")
print(result)
```

value is the parameter. strip() returns text without outer whitespace. return passes that result to the caller. The call assigns it to result and prints Prepare slides. The dot calls a method on a value, a notation you already use in Java.

Default values and named arguments

```
def title_required(partial=False):
    return not partial

print(title_required())
print(title_required(partial=True))
```

partial defaults to False when the caller omits it. partial=True names the parameter being set; it is not a variable assignment in the caller. The two printed results are True and False. not inverts a boolean, like ! in Java and C.

Import code from another file

```
from task_api.validation import validate_task
```

import makes a supplied function available under its name. A Python file is a module; task_api is the package folder, and validation is its validation.py module. Inside that package, from .validation import validate_task refers to a sibling module.

You will see __init__.py and __name__ in the supplied setup. Keep them as provided. Today you call the existing store object and ApiError class; you do not need to implement Python classes.

Checkpoint In warm up sections D and E, identify a definition, a call, a return value and a named argument.

Step 3

Read one working Flask route

Open task_api/tasks.py and find get_task. It already works. A route connects an HTTP method and URL to the Python function that handles the request.

```
@bp.get("/<int:task_id>")
def get_task(task_id):
    item = store().get(task_id)
    if item is None:
        raise ApiError(404, "not_found", "Task not found")
    return jsonify(item)
```

Line or expression	Read it this way
@bp.get(...)	Register the function below for this GET route
<int:task_id>	Extract an integer from the URL into task_id
store().get(task_id)	Call store(), then get this task from that object
item is None	No task was found; None is the no-result marker
raise ApiError(...)	Raise the supplied application exception
return jsonify(item)	Send this task as a JSON response

The line beginning with @ is a decorator. Here Flask uses it to register a route. Keep it immediately above the function definition. It looks like an annotation in Java, but this Flask operation attaches the Python function to the route.

```
python api_client.py GET /api/tasks/999
```

Expected: 404 with an error body. For /api/tasks/1, task_id is 1 and the supplied route returns 200. Both requests reached a running server; their target resources differ.

Keep store.py, errors.py, __init__.py, api_client.py and verify_api.py as supplied. The core edits are only validation.py and tasks.py.

Understand Blueprint and the app factory

The starter groups task routes into a Blueprint. A Blueprint records routes to register on a Flask application. The app factory is a function that creates and configures that application.

```
# In task_api/tasks.py
bp = Blueprint("tasks", __name__, url_prefix="/api/tasks")

@bp.get("")
def list_tasks():
    return jsonify(store().list())
```

tasks is the group name used in internal endpoint names. url_prefix adds /api/tasks to each local route. The empty local path "" therefore exposes GET /api/tasks; /<int:task_id> exposes an item under that prefix.

```
# In task_api/__init__.py, inside create_app()
app = Flask(__name__)
# Other supplied setup creates the store and error handlers.
app.register_blueprint(bp)
# The factory returns app after all supplied setup.
```

Name	What it is	Its role here
app	A Flask application object	Handles the registered requests
create_app	A Python function	Builds and returns a configured app
bp	A Blueprint object	Groups the task route registrations

The group must be registered before the app handles requests. The factory already does this. Registering this Blueprint does not create a separate server or a microservice.

For an internal endpoint, Flask combines the Blueprint name and function name: tasks.get_task. The URL is a separate concept. Later, url_for("tasks.get_task", task_id=4) builds /api/tasks/4.

Checkpoint Find the actual registration line in the starter. Keep it in place. The optional comparison later in this guide shows the same public route with and without a Blueprint.

Step 4

Plan validation before storing data

Lina sends a proposed task title. A JSON parser can read a number or a string, but our application accepts only a string title. Validation checks the application rules after JSON has been parsed.

Input	Rule for this API
Body	A JSON object, represented by a Python dict
title	String of 3 to 120 characters after stripping outer whitespace
done	A real boolean; accept True and False
id	Server managed; reject it in client input
Any other key	Reject it as an unknown field

```
def validate_task(payload, partial=False):
    # Supplied checks and the two TODO blocks are inside this function.
    clean = {}
    problems = {}
```

payload holds the parsed input. clean will hold only accepted writable fields. problems will hold messages under field names. An empty dictionary is written {}. Assignment such as clean["title"] = value adds an entry.

Call	Purpose	What an omitted title means
validate_task(payload)	Creation mode	Record a missing title problem
validate_task(payload, partial=True)	Partial update mode	Leave title out of clean

The validator returns clean only after every supplied field has passed. It has no call to store(). Creating or updating storage belongs to the route handler.

Open validation.py. This edition supplies the body check, unknown-field loop, mode checks and final decision. Your first edits are the two small blocks marked TODO 1a and TODO 1b.

Predict Why should {"title":42} and {"title":"Example","done":"false"} fail even though they are valid JSON? Write one application rule for each.

Read the supplied checks and exceptions

Read the supplied statements before the TODO blocks. They collect feedback using Python operations you can recognize from their role.

```
allowed_fields = ["title", "done"]
for key in payload:
    if key not in allowed_fields:
        if key == "id":
            problems[key] = "Read-only field"
        else:
            problems[key] = "Unknown field"
```

allowed_fields is a list, written with square brackets. for key in payload visits the dictionary keys. For {"title":"Example","id":99}, the keys are title and id. not in tests absence; == compares values. The id key adds feedback to problems.

```
if partial and not payload:
    problems["body"] = "Supply at least one writable field"
if not partial and "title" not in payload:
    problems["title"] = "Title is required"
```

and plays the role of && in Java and C. An empty dictionary is false in a condition; a nonempty one is true. In partial mode, an empty body has no requested change. In creation mode, title must be present.

How a reported problem becomes an HTTP error

```
if problems:
    raise ApiError(400, "validation_error", "Invalid task data", problems)
return clean
```

if problems means at least one message was collected. raise is similar to throw in Java: normal execution leaves this call, so return clean is skipped. The supplied Flask error handler catches ApiError and creates the HTTP error response.

The supplied class expects a status, code, message and optional field feedback. In an HTTP request, a rejected input gives a JSON error. Calling the function directly without catching its exception gives a Python traceback instead.

Checkpoint Which two dictionaries have different jobs? Which line stops a caller from receiving partially accepted input?

Complete the title check

Open validation.py. TODO 1a sits inside if "title" in payload, after value = payload["title"]. Replace the two TODO 1a lines, the comment and its 501 exception, with this block. Keep the supplied enclosing if and value assignment.

```
if not isinstance(value, str):
    problems["title"] = "Title must be a string"
else:
    value = value.strip()
    if len(value) < 3 or len(value) > 120:
        problems["title"] = "Use 3 to 120 characters after trimming"
    else:
        clean["title"] = value
```

The indentation above is the indentation to use in the file. The first if not isinstance line has eight leading spaces, the same level as the existing value assignment. Its nested statements use twelve spaces; the innermost assignments use sixteen. Each else aligns with its own if.

Expression	Meaning
isinstance(value, str)	Is value a string
not ...	Use the opposite boolean result
value.strip()	Return text without surrounding whitespace
len(value)	Count the string characters
A or B	Accept either condition, like ||
clean["title"] = value	Keep the accepted normalized title

Check type before strip(): a number such as 42 has no strip method. Check length after stripping so that spaces alone do not become a title.

Predict For " Prepare project proposal ", which value enters clean? For 42, which branch records feedback? No storage operation runs in this block.

Complete the boolean check

TODO 1b is inside if "done" in payload, after value = payload["done"]. Write a short if and else using the rules below. Keep that supplied presence check and value assignment.

The Python test you need

```
type(value) is not bool
```

Read this as: the exact type of value is not bool. type reports a value type; bool is the boolean type. This API rejects numbers 0 and 1, strings and None. It accepts both True and False.

Supplied value in Python	Expected decision
True	Accept and copy to clean
False	Accept and copy to clean
"false" or "yes"	Record a done problem
0 or 1	Record a done problem
None	Record a done problem

Write these two branches

- If the type is not bool, assign "Done must be a JSON boolean" to problems["done"].
- Otherwise, assign value to clean["done"].

Use the title check as your indentation model. Replace the TODO 1b exception. The final supplied if problems and return clean stay outside both field blocks, at the function indentation level.

A common mistake to explain

```
payload = {"done": False}
print("done" in payload)
print(payload["done"])
```

This prints True then False. Do not replace the key-presence check with if value. Do not convert arbitrary input with bool(value): bool("false") is True because the string is nonempty.

Checkpoint Show a neighbour which condition lets False reach clean. Then save validation.py.

Check validation before writing routes

Run the supplied small checker in Terminal B. It calls your validator with Python values; it does not send HTTP requests and does not change stored tasks.

```
python check_validation.py
```

It tests 10 cases. After completing both TODO blocks, every case should print PASS and the last line should be Validation checks passed: 10 / 10. A 501 in a failure message identifies an unfinished TODO.

Values to inspect in your own code

Input and mode	Expected result or feedback
{"title":" Prepare project proposal "}	{"title":"Prepare project proposal"}
{"done": False}, partial=True	{"done": False}
{"title": 42}	400 validation error for title
{"done": 0}, partial=True	400 validation error for done
{}, partial=False	400 because title is required
{}, partial=True	400 because no change was supplied
{"title":"Example","id":99}	400 because id is read only

These are Python dictionary examples, so False is capitalized. In a JSON payload file it would be false. The checker catches expected validation exceptions and prints a short result instead of a traceback.

Connect this function to the next step

A successful validator call returns a dictionary. A rejected call raises ApiError and does not return clean. The POST handler can therefore call the store only after validate_task has returned.

Checkpoint Explain why {"done": False} is not the same input as {}. In notes.md, write the two jobs of clean and problems. Continue once the small checker passes.

Step 5

Follow the guided POST handler

Open tasks.py and replace the TODO 2 function with this guided code. Keep one copy of its decorator. This handler connects input, validation, storage and an HTTP response.

```
@bp.post("")
def create_task():
    payload = request.get_json()
    fields = validate_task(payload)
    item = store().create(fields)
    response = jsonify(item)
    response.status_code = 201
    response.headers["Location"] = url_for(
        "tasks.get_task", task_id=item["id"])
    return response
```

Statement	What it contributes
request.get_json()	Read the HTTP JSON body as Python values
validate_task(payload)	Return accepted fields or raise an error
store().create(fields)	Create the task and assign its ID
jsonify(item)	Build a Flask response containing JSON
response.status_code = 201	Set the successful creation status
response.headers["Location"] = ...	Point the client to the new task URL

item is a task dictionary. response is an HTTP response object, with a body, status and headers. A dot can access a property such as status_code as well as a method such as get_json().

url_for receives the internal endpoint tasks.get_task. The named argument task_id=item["id"] supplies the new ID. For a new ID of 4, it builds /api/tasks/4.

Checkpoint Point to the first line that changes storage. If validate_task raises an error, does execution reach that line? Explain before sending the request.

Create and retrieve the proposal task

Save both edited files. In Terminal A press Ctrl+C, then start Flask again with the same --debug command. This makes the starting store clear. A reload or restart recreates the three seed tasks.

```
python -m flask --app task_api run --debug
```

Send the first creation in Terminal B

```
python api_client.py POST /api/tasks --json payloads/create.json
```

--json tells the supplied client to read that JSON file and send it with the JSON Content-Type. Open payloads/create.json to see the outer spaces around the proposed title.

Expected HTTP result in a fresh store

```
HTTP 201
Location: /api/tasks/4
{"id": 4, "title": "Prepare project proposal", "done": false}
```

Your client display also shows other headers. The JSON key order can differ. The store assigned id 4 and defaulted done to false; the validator removed the outer title spaces.

```
python api_client.py GET /api/tasks/4
```

Expected: 200 with the same task. Record the actual ID and Location from your POST. If the returned ID is not 4, use the actual Location path in later commands.

Check a rejected creation

```
python api_client.py POST /api/tasks --json payloads/invalid_title.json
python api_client.py GET /api/tasks
```

invalid_title.json contains a numeric title. Expected: 400, with no task added. The following collection GET still contains only the seeds and the successful proposal task.

```
python -m unittest verify_api -k create_location
```

-k selects tests whose names contain the following word. This creation check needs no server and should end with OK. Copy your real creation and retrieval output into evidence.txt. The checker application is separate from the live server.

Step 6

Complete a partial update

Lina now sends only {"done":true}. PATCH changes supplied fields in this API. The title is omitted, so it must stay out of the validated update dictionary.

Read the supplied start of update_task

```
@bp.patch("/<int:task_id>")
def update_task(task_id):
    item = store().get(task_id)
    if item is None:
        raise ApiError(404, "not_found", "Task not found")
    payload = request.get_json()
    # TODO 3 goes here.
```

None means the target was not found. It is different from a valid done value of False. The supplied order checks target existence before reading the request body.

Replace TODO 3 with three statements

- Assign fields from validate_task(payload, partial=True). The named argument activates the mode you read earlier.
- Assign item from store().update(task_id, fields). Both task_id and fields are arguments to the method.
- Return jsonify(item). With this ordinary response, Flask uses status 200.

The store update copies and merges the accepted fields into the existing task. For a clean dictionary containing only done, the existing title is retained. You do not write the store merge today.

Prepare the task again after editing

Save tasks.py, stop and restart Terminal A, then run these in Terminal B. A code reload may have removed the task created before your edit.

```
python api_client.py POST /api/tasks --json payloads/create.json
python api_client.py GET /api/tasks/4
```

Follow the actual returned Location. Continue once the proposal task exists with done false. Avoid code edits between the two experiments on the next page.

Compare accepted and rejected updates

First complete the existing task

```
python api_client.py PATCH /api/tasks/4 --json payloads/complete.json
python api_client.py GET /api/tasks/4
```

Expected: PATCH returns 200. The GET has the same id and title with done true. The complete.json file supplies only done.

```
{"id": 4, "title": "Prepare project proposal", "done": true}
```

Then send two fields with one invalid value

Open payloads/invalid_mixed.json. Its title Should not be saved is valid, but done is the string "yes". Predict the entire task after this request before running it.

```
python api_client.py PATCH /api/tasks/4 --json payloads/invalid_mixed.json
python api_client.py GET /api/tasks/4
```

Observation	Expected result
Rejected PATCH	400 with feedback for done
Title after the following GET	Prepare project proposal
done after the following GET	true, from the earlier accepted PATCH

The valid title may enter the temporary clean dictionary, but the final if problems raises before returning it. The handler therefore does not call update. Compare all task fields before and after rejection, not the X-Request-ID header, which labels each request.

```
python -m unittest verify_api -k patch_keeps
python -m unittest verify_api -k patch_invalid_fields
```

Each -k filter selects the matching test name. Both targeted checks should end with OK.

Checkpoint Which argument permits an omitted title? Which line stops a partly valid update from changing storage? Save the accepted and rejected request evidence.

Step 7

Complete and observe deletion

Lina no longer needs the proposal task. In tasks.py replace TODO 4 using the following sequence. The delete helper already exists in the store.

Write the handler in three parts

- Call store().delete(task_id) and assign its result to deleted.
- If deleted is False, raise ApiError(404, "not_found", "Task not found"). Follow the missing-target check in get_task as your indentation model.
- Otherwise let execution reach return "", 204.

The store returns True when it removes a task and False when no task exists. The return form "", 204 gives Flask a pair: an empty body and a status code. It is not a JSON object and is not the C comma operator.

Restore a known state after the code edit

Save the handler. Stop and restart Terminal A, then run this sequence in Terminal B. Use the actual Location returned by creation.

```
python api_client.py POST /api/tasks --json payloads/create.json
python api_client.py PATCH /api/tasks/4 --json payloads/complete.json
python api_client.py DELETE /api/tasks/4
python api_client.py GET /api/tasks/4
python api_client.py DELETE /api/tasks/4
```

Request after the task exists	Status	Body
First DELETE	204	Empty
GET after removal	404	JSON error
Repeated DELETE	404	JSON error

```
python -m unittest verify_api -k delete_empty_body
```

The client displays <empty body> for 204; that text is its display label, not bytes sent by the server. Both DELETE attempts leave the task absent. That same intended effect is what idempotence means here, even though response statuses differ.

Step 8

Record the uninterrupted task journey

Now that the code is complete, collect one clear sequence. Save all files, stop Terminal A and start a fresh server without --debug. Do not edit or restart it during this experiment.

```
python -m flask --app task_api run
```

In Terminal B, run the following commands in order. A fresh store first creates task 4. Use your actual returned Location if it differs.

```
python api_client.py POST /api/tasks --json payloads/create.json
python api_client.py GET /api/tasks/4
python api_client.py PATCH /api/tasks/4 --json payloads/complete.json
python api_client.py GET /api/tasks/4
python api_client.py PATCH /api/tasks/4 --json payloads/invalid_mixed.json
python api_client.py GET /api/tasks/4
python api_client.py DELETE /api/tasks/4
python api_client.py GET /api/tasks/4
```

Request	Status	What the task data shows
1 Create	201	Trimmed proposal title and done false
2 Retrieve	200	Same newly created task
3 Complete	200	Same title and id, done true
4 Retrieve	200	Completed proposal task
5 Invalid mixed update	400	Feedback; no accepted change
6 Retrieve	200	Exactly the same task data as request 4
7 Delete	204	Empty body; task removed
8 Retrieve	404	Task is absent

Copy the client output, including the creation Location, into evidence.txt. If a code correction is needed, finish the edit first, restart, then record the sequence again from creation.

Check and explain your work

Run the complete supplied checker

```
python verify_api.py
```

For the completed implementation, expect 20 test methods and OK, with additional input cases inside some tests. These tests use fresh applications and do not modify the live task used for your HTTP evidence.

Copy the full output into checker_output.txt. A FAIL identifies an expected result that differed; ERROR usually means an unexpected exception. Read the reported test name, file and last error line before editing.

Write short explanations in notes md

- Why is {"title":42} valid JSON but invalid task data in this API?
- For {"done": False}, what does the presence test return, and why must False enter clean?
- How does partial=True affect an omitted title?
- If a valid title accompanies invalid done, why does neither stored value change?
- What does return "", 204 tell Flask, and why can the next DELETE return 404?

Use your own words and refer to a condition, a statement or an observed response. A neighbour may help you debug, but each student should be able to explain one handler and one rejected input.

Submit a small source archive

Include task_api, requirements.txt, the supplied client and unchanged main checker, payload files, notes.md, evidence.txt and checker_output.txt. Keep a README with your name and the run commands. Include your warm up edits if you made them.

Exclude .venv, __pycache__ and temporary files. If a required check still fails, record that failure with the steps needed to reproduce it.

Core completion The baseline works, validation checks pass, POST creates with 201 and Location, PATCH preserves omitted fields and rejects invalid input, and DELETE returns an empty 204 for an existing task.

Reference Read Python errors and HTTP results

A Python error and an HTTP error are different observations. First identify where the message appeared: the editor, Terminal A, a checker, or the client output.

What you see	First thing to inspect
IndentationError or TabError	Four spaces per block level; avoid mixed tabs and spaces
SyntaxError	A missing colon, quote, parenthesis or bracket near the reported line
NameError mentioning true or false	Use True and False in Python; lowercase belongs in JSON
ModuleNotFoundError for flask	Correct virtual environment and requirements installation
KeyError for a dictionary key	Read a key only after checking whether it is present
AttributeError mentioning strip	Check the title type before calling a string method
Connection failure in the client	Terminal A is running and the expected port is available

HTTP status	Meaning in this lab
400	Read JSON syntax, object shape or field feedback
404	Check the path, actual task ID and any store reset
405	The main API has no handler for that method, including PUT
415	The request must use JSON Content-Type
501	The starter still contains a TODO exception

For a traceback, start with the last line to identify the error, then find the relevant file and line above it. An exception during a direct Python call is not itself an HTTP response.

If port 5000 is occupied

```
python -m flask --app task_api run --debug --port 5001
python api_client.py GET /api/health --base http://127.0.0.1:5001
```

Run the server command in Terminal A and the client command in Terminal B. If using 5001, append --base http://127.0.0.1:5001 to every later api_client.py command too.

When asking for help, show your step, command, expected result, actual response or last traceback lines, and the function you edited. This gives a neighbour or the instructor a specific issue to inspect.

Reference Python notation for Java and C students

Purpose	Java or C reminder	Python used in this lab
A named value	Type declarations such as String or int	title = "Prepare slides"
A block	Braces around statements	Colon followed by indentation
A function	Function or method declaration	def name(parameter):
True and false	true and false in Java	True and False
Missing object or result	null or NULL in familiar contexts	None; test with is None
Boolean operations	&& || !	and or not
Equality	== for primitive values in Java or C	== compares values; = assigns
Key and value data	A Java Map is a useful analogy	{"title": "Example"} is a dict
Key presence	A map key lookup operation	"title" in payload
A list of values	A list or array is a useful analogy	["title", "done"]
An exception	throw in Java	raise ApiError(...)
Return to caller	return	return clean

These are reading aids for this exercise, not exact equivalences between the languages. Use is None for the missing-result test and == for ordinary value comparisons. Do not use is to compare string contents.

Distinguish the three places you work

Place	What belongs there
Terminal	python python_warmup.py and other run commands
A .py file in the editor	Python definitions, conditions and statements
A .json payload file	JSON data with true, false and null

The code examples in this tutorial do not require the interactive >>> prompt. Use the supplied programs and ordinary terminal commands.

Extension A Compare direct routes and Blueprints

Complete this comparison after the core workflow or after class. It uses separate small applications with an empty task list. It does not change the main task_api or its seeded store.

Version	Registration approach	Intended public path
Direct app route	@app.get("/api/tasks")	/api/tasks
Blueprint route	Prefix /api/tasks plus @bp.get("")	/api/tasks

Open exercises/blueprint_direct.py, blueprint_routes.py and blueprint_grouped.py. Each create_app() returns an application for the observer. The direct version works; the grouped version has two deliberate exercise defects.

```
python -m exercises.observe_blueprint
```

The -m command runs the named module inside the exercises package. The supplied observer calls Flask applications inside Python, so Terminal A is not needed for this comparison.

Repair one issue at a time

- Predict why GET /api/tasks is initially 404 for the grouped version. In blueprint_grouped.py complete TODO B by registering the imported bp before return app. Use the registration example from Step 3.
- Run the observer again. Read the actual URL map: the prefix is now repeated in the task path. The intended /api/tasks request still returns 404 at this intermediate stage.
- In blueprint_routes.py complete TODO A by choosing a local path that does not repeat /api/tasks. Run again: both versions should return 200 and [] at /api/tasks. /api/health still works.

Explain the two differences

The direct endpoint is list_tasks. The registered group endpoint is tasks.list_tasks. Explain why both url_for calls can build the same public URL, and why the grouped application needs a registration line.

Record the observations in separate comparison notes. A Blueprint groups registrations inside an app. create_app is the factory function that builds the app; it is not a competing route syntax.

Extension B Compare PUT and PATCH

Use the independent, completed comparison application in exercises/put_patch_demo.py. You do not need to implement this helper. It accepts both methods at /demo/tasks/4; the main /api/tasks API still returns 405 for PUT.

Read this comparison contract first

PUT supplies the replacement writable state, so this exercise requires both title and done. PATCH sets only supplied writable fields and retains omitted ones. Both validate before changing data; the server keeps id 4.

An incomplete PUT is rejected by this documented contract. Do not assume that every API silently deletes or defaults a field omitted from a PUT.

```
{"id": 4, "title": "Prepare project proposal", "done": false}
```

Predict four independent results

Each row starts from the original task above. Predict the status and the complete task after a following GET. Do not carry a previous row result into the next row.

Case	Method	JSON body
A	PATCH	{"done":true}
B	PUT	{"done":true}
C	PUT	{"title":"Revised project proposal", "done":true}
D	PATCH	{"title":"Revised project proposal"}

```
python -m exercises.observe_put_patch
```

The observer prints a before state, request, status, response and after state for every independent case. It needs no server. Compare the output with your predictions and explain the difference between A and B.

Finally, the observer repeats a valid PUT and a PATCH that sets done true. Both particular requests have the same final representation when repeated. Does that show that every possible PATCH instruction is idempotent, or only that this field-setting one is?

Extension C Observe the store after restart

Use the live server after the uninterrupted core evidence sequence, where the proposal task was created and deleted. Lina now creates Prepare slides so that a still-existing task can be observed across a restart.

Create another task

The updated starter supplies payloads/next_task.json with {"title":"Prepare slides"}. Run these in Terminal B and record the actual returned Location.

```
python api_client.py POST /api/tasks --json payloads/next_task.json
python api_client.py GET /api/tasks
```

If the proposal was the only earlier successful creation in this instance, the new ID is 5. Task 4 was deleted, but deletion did not rewind the next-ID counter. Use the actual returned ID if your request history differs.

Predict before restarting

Write the current task IDs in notes.md. Predict whether the new task will still exist. Stop Terminal A with Ctrl+C, then start the same application again.

```
python -m flask --app task_api run
```

In Terminal B retrieve the collection and the recorded path of Prepare slides.

```
python api_client.py GET /api/tasks
python api_client.py GET /api/tasks/5
```

Observation for the stated sequence	Expected result
Before restart	IDs 1, 2, 3 and 5
Collection after restart	Original seed IDs 1, 2 and 3
GET of task 5 after restart	404

The supplied TaskStore keeps dictionaries and an ID counter in process memory. create_app builds a fresh store when the application restarts. The new task data disappears because there is no database persistence in this lab.

Explain Which module would need a different implementation to persist tasks? Which request paths and validation rules could still be retained? Keep this observation separate from the core evidence sequence.

Extension D Inspect request boundary errors

Run these after recording the core story. For every rejected creation, compare the collection before and after so that a status code is connected to stored data.

```
python api_client.py GET /api/tasks
python api_client.py POST /api/tasks --raw payloads/malformed.txt
python api_client.py POST /api/tasks --json payloads/array.json
python api_client.py POST /api/tasks --json payloads/create.json --content-type text/plain
python api_client.py POST /api/tasks
python api_client.py GET /api/tasks
```

Experiment	Status	Where it is rejected
Malformed JSON text	400	JSON parsing
Valid JSON array	400	Application body type check
JSON sent as text/plain	415	Request media type check
Bare POST from this client	415	No JSON Content-Type

Use --raw for malformed JSON. The --json option parses the file in the client first, so it would reject malformed syntax before sending an HTTP request. A long command that wraps here is still one terminal line.

```
python api_client.py PUT /api/tasks/1 --json payloads/create.json
```

The main API returns 405 with an Allow header listing supported methods. This differs from the separate PUT and PATCH comparison application. Keep the supplied error handler and main checker unchanged.

Official references for further reading

Python control flow and functions

Python data structures

Python errors and exceptions

Flask quickstart and request handling

Flask Blueprint registration

PATCH semantics RFC 5789

The starter and this tutorial define the teaching API field rules and operation order. The linked documentation explains the Python, Flask and HTTP mechanisms used in those examples.